Downshift: Project Documents

Nebius x NVIDIA Global AI Hackathon | Coding and Agentic Engineering track | Deadline: 30 Oct 2026, 10:30 pm IST | Owner: Cel

Contents: 1 One-Page Spec, 2 Product Requirements, 3 Technical Requirements, 4 Architecture, 5 System Design, 6 Database Design, 7 Coding Standards and Tools, 8 Evaluation Plan, 9 Demo Script, 10 Submission Checklist, 11 Access Checklist, 12 Build Schedule.

1. One-Page Spec

Problem. Teams run AI agents on their most capable, most expensive model for every step, including trivial ones. Nobody knows which steps could safely run on a cheaper model, so they either overpay or downgrade blindly and break quality.

Solution. Downshift records an agent run, replays each step on cheaper Nemotron tiers, scores the results, and produces a per-step routing config. It then re-runs the agent with that config to prove the savings end to end.

Target user. Developers and small teams running agents on Nemotron through Nebius Token Factory.

Promise. "Your agent doesn't need Ultra for every step."

In scope.

- Python SDK wrapper that records calls from any OpenAI-compatible client
- Step-level replay on Nemotron Super and Nano
- Objective checks plus an LLM judge
- Routing config export and a router middleware
- End-to-end validation run
- Web UI: run list, timeline, step diff, cost chart

Non-goals. Non-OpenAI-compatible APIs, multi-user auth, live production traffic routing, fine-tuning, mobile app, databases beyond SQLite.

Success definition (targets, to be confirmed by measurement). On at least 2 real agents: 30% or more cost reduction while keeping 95% or more of baseline task success. If the measured numbers are lower, report them honestly. Do not inflate.

2. Product Requirements (PRD)

Goals

- Make model-tier decisions evidence-based instead of guesswork.
- Show real, measured savings in a 3-minute demo.
- Showcase Nemotron tiering and Token Factory economics.

Persona. Dev, an engineer running a multi-step agent on Nemotron 3 Ultra. Dev's bill is high and Dev suspects most steps do not need Ultra, but has no safe way to test that.

User stories

- As Dev, I add one line to my agent so every model call is recorded.
- As Dev, I see my run as a timeline with cost per step.
- As Dev, I replay steps on cheaper tiers and see where quality holds.
- As Dev, I export a routing config and re-run to confirm savings.

Functional requirements

- FR-1: Wrap an OpenAI-compatible client with one line to record calls.
- FR-2: Store per step: messages, tool definitions, response, tool calls, tokens, latency, cost, model.
- FR-3: Group steps into runs, tagged by task and by step tag.
- FR-4: Replay selected steps on Super and Nano in parallel.
- FR-5: Objective checks: valid tool-call schema, same tool name, matching arguments, task-specific tests where available.
- FR-6: LLM judge with a fixed rubric for free-text steps.
- FR-7: Per-step recommendation: the cheapest tier that meets the quality threshold, labeled "safe to downgrade" or "keep on Ultra".
- FR-8: Export a routing config (YAML) and provide a router middleware that applies it.
- FR-9: Validation run with the router, compared with baseline on cost, task success and latency.
- FR-10: UI with run list, timeline colored by tier, step detail diff, cost chart, export button.
- FR-11: Demo mode with a pre-recorded run so the demo never depends on live luck.

Non-functional requirements

- Replay of a 30-step run completes in under 5 minutes.
- No API keys in the repository. Setup from README in under 10 minutes.
- Retries with backoff on API errors. Clear error messages.
- UI loads in under 2 seconds on the demo dataset.

Success metrics. Cost reduction %, quality retained %, % of steps downgraded, judge-versus-objective agreement, time to first insight.

Risks

- Judge bias or error: mitigated by objective checks first and blind A/B judging.
- Savings too small on the test agents: choose agents with a mix of easy and hard steps, and report honestly.
- Credit exhaustion: cap steps and runs (see Evaluation Plan).
- Scope creep: the non-goals list is binding.

Open questions to verify on day 1. Exact Nemotron model IDs on Token Factory, API base URL and rate limits, per-model pricing, and whether Serverless Jobs are available on your credits.

3. Technical Requirements

Mandatory (hackathon rules)

- Runs on Nebius Token Factory or AI Cloud and uses at least one NVIDIA open source model (Nemotron).
- Public repository with an open source license (MIT) visible at the top, plus a README with setup and run instructions.
- Working demo URL and a public YouTube video of 3 minutes or less.
- Written feedback on Nebius and NVIDIA tools, and a note on where Token Factory helped.

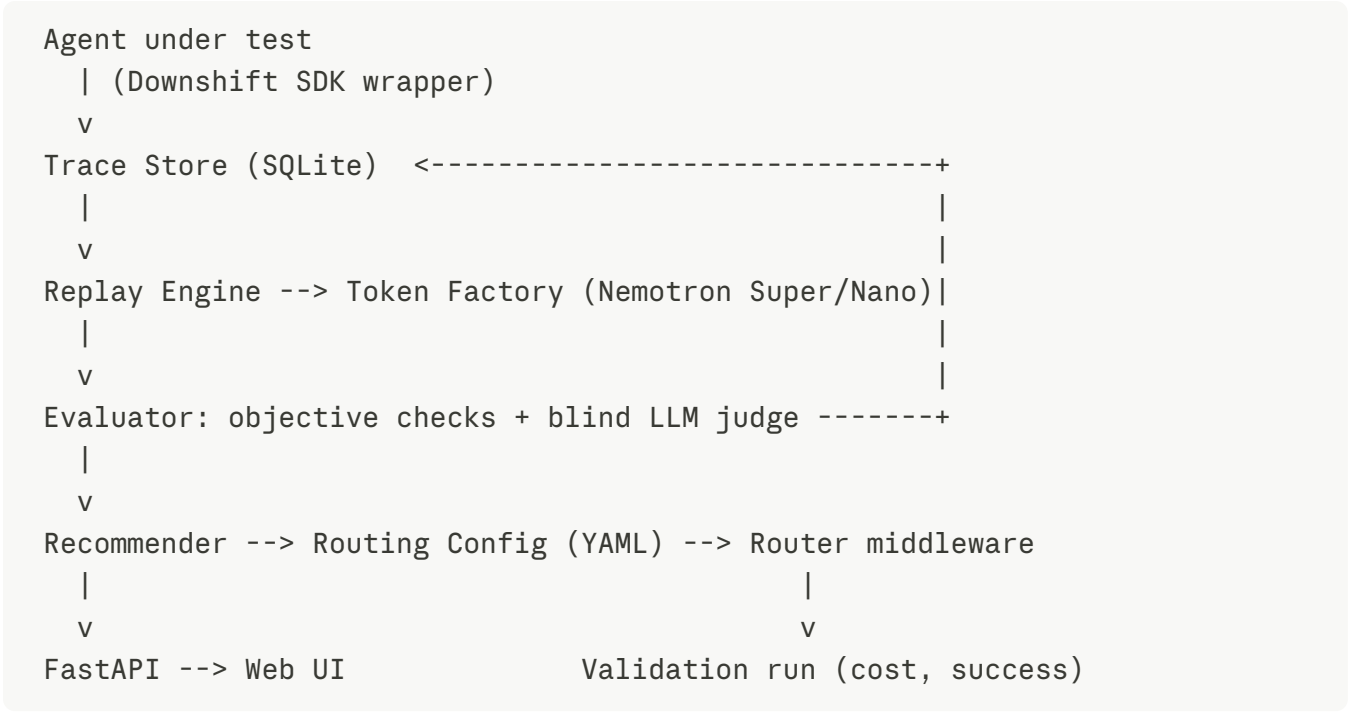
Technical requirements

- Python 3.11 or newer. FastAPI backend. httpx for async calls. Pydantic v2 for schemas.
- OpenAI-compatible client against the Token Factory endpoint. Confirm the base URL in the Nebius docs before coding.
- Model IDs, prices and thresholds live in config, never hardcoded.
- Replay uses temperature 0 where supported. Record all sampling parameters.
- Concurrency limit (semaphore) and retry with backoff on 429 and 5xx errors.
- SQLite for storage. React and TypeScript for the UI.
- Docker image for deployment. Hosted demo on Nebius Serverless Endpoints if accessible (encouraged by the rules, not required), otherwise a simple free host.
- Replay offloaded to Nebius Serverless Jobs only after the asyncio version works.

Constraints

- Solo builder, 5 build days, limited credits.
- Hard caps: 40 steps per run, 5 baseline runs per test agent.

4. Architecture



Components

- **SDK wrapper:** intercepts chat-completion calls, records them, forwards them unchanged.
- **Trace store:** one source of truth for runs, steps, replays, evaluations.
- **Replay engine:** re-sends each recorded step's exact context to candidate tiers, in parallel.
- **Evaluator:** objective checks first, then a blind judge for free-text output.
- **Recommender:** picks the cheapest passing tier per step tag.
- **Router middleware:** applies the config at call time.
- **Validation runner:** re-runs the full agent with the router and compares with baseline.
- **API and UI:** read-only views plus triggers for replay and export.

Model roles

- Nemotron 3 Ultra: baseline agent, and judge for free-text steps.
- Nemotron Super and Nano: candidate tiers.

Key decisions

1. **Step-level replay, proven by an end-to-end run.** Step-level replay is cheap but can overestimate quality, because the replayed step sees Ultra's earlier outputs. The validation run is the real proof.
2. **Objective checks before LLM judging.** They are deterministic and cannot be argued with.

3. **Blind A/B judging.** The judge does not know which output came from which model, and the order is randomized, to reduce self-preference bias.
4. **Routing by step tag, not step index.** Index does not generalize to new runs.
5. **asyncio first, Serverless Jobs second.** Do not risk the core on an optional service.
6. **Demo mode.** A saved real run is bundled so the demo always works.

5. System Design

Flow

1. **Record:** the agent runs on Ultra through the wrapper. Each call is stored as a step with its tag.
2. **Replay:** for each step, enqueue one replay per candidate tier. Workers call Token Factory with the stored messages and tools.
3. **Evaluate:** run objective checks. For free-text steps, call the judge with the rubric and both outputs in random order. Store the score and reason.
4. **Recommend:** for each tag, choose the cheapest tier whose pass rate meets the threshold (default 0.9, adjustable in UI).
5. **Export:** write the routing config.
6. **Validate:** re-run the agent with the router. Store cost, success and latency.

Recommendation rule. For each step: if the objective check fails, the tier scores 0. Otherwise the tier scores the judge score (0 to 1). Per tag, choose the cheapest tier where the mean score is at or above the threshold and no objective failures occurred. Tool-call steps require an objective pass. If no cheaper tier passes, keep Ultra.

Routing config example

```
version: 1
default: ultra
threshold: 0.9
routes:
  - tag: search_query
    model: nano
  - tag: summarize_page
    model: super
  - tag: final_answer
    model: ultra
```

API endpoints

- POST /runs (create), GET /runs, GET /runs/{id}
- POST /runs/{id}/replay (start replays), GET /runs/{id}/replay/status

- GET /runs/{id}/report (recommendations, savings)
- POST /runs/{id}/config (generate and export routing config)
- POST /runs/{id}/validate (start validation run)

Failure handling

- API error: retry with exponential backoff, then mark the replay as errored (never as passed).
- Judge returns invalid JSON: retry once, then mark the evaluation as unscored and exclude from averages.
- Credit cap reached: stop replays and show a clear message.

Security

- Keys only in environment variables. Provide .env.example.
- Redact secrets and obvious personal data from stored payloads.
- Do not log full prompts to the console.

6. Database Design

SQLite through SQLModel. All raw payloads are stored as JSON for reproducibility.

- **runs:** id, task_name, agent_name, baseline_model, status, started_at, total_cost_usd, tokens_in, tokens_out, task_success (nullable boolean), meta (JSON)
- **steps:** id, run_id (FK), idx, tag, kind (tool_call or text), model, request (JSON), response (JSON), tool_calls (JSON), tokens_in, tokens_out, latency_ms, cost_usd, created_at
- **replays:** id, step_id (FK), tier, model, response (JSON), tool_calls (JSON), tokens_in, tokens_out, latency_ms, cost_usd, error, created_at
- **evaluations:** id, replay_id (FK), objective_pass (boolean), objective_detail (JSON), judge_score (real, nullable), judge_reason, judge_model, created_at
- **recommendations:** id, run_id (FK), tag, recommended_tier, mean_score, savings_usd, label, created_at
- **routing_configs:** id, run_id (FK), threshold, yaml (text), created_at
- **validations:** id, run_id (FK), config_id (FK), cost_usd, task_success, quality_retained (real), latency_ms, created_at
- **model_prices:** model, input_per_mtok, output_per_mtok, source, updated_at

Relationships. runs 1-to-many steps; steps 1-to-many replays; replays 1-to-1 evaluations; runs 1-to-many recommendations, configs, validations.

Indexes. steps(run_id, idx), replays(step_id, tier), evaluations(replay_id).

Notes. Prices come from the model_prices table, filled from the official Nebius pricing page at build time (do not guess). Back up exported logs to your Google Drive. Do not store personal data.

7. Coding Standards and Tools

Tools

- Python 3.11, uv (dependencies), FastAPI, Pydantic v2, SQLModel, httpx, tenacity (retries)
- Testing: pytest and pytest-asyncio
- Quality: ruff (lint and format), mypy (type checking), pre-commit hooks
- Frontend: Vite, React, TypeScript (strict mode), Tailwind, Recharts, ESLint, Prettier
- Delivery: Docker, GitHub Actions (lint and tests on every push)
- Research agent search: Tavily (also targets the Best Use of Tavily prize)
- Colab: scratch analysis and judge-prompt experiments only. Google Drive: log backups and demo assets.

Standards

- Type hints on every function. mypy must pass.
- ruff must pass with no warnings. One formatter, no manual style debates.
- Functions do one job and stay under about 40 lines. No dead code, no commented-out code.
- All config through environment variables or a single config file. No hardcoded model IDs, prices or keys.
- Never commit secrets. Ship .env.example. Add .env to .gitignore before the first commit.
- Every external call has a timeout, retry with backoff, and a typed error.
- Structured logging (JSON) at INFO level. Never log API keys or full prompts.
- Tests for the core logic: recommendation rule, objective checks, cost calculation, routing config parsing. Do not chase coverage numbers; test what can break the demo.
- Git: small commits, conventional messages (feat:, fix:, docs:, test:), work on feature branches, merge to main only when tests pass.
- AI-assisted code: you read and understand every line before committing. Judges may ask.
- Every public function and module gets a one-line docstring. README stays current as you build, not at the end.

Repository layout

```
downshift-agent-router/
  LICENSE            (MIT)
  README.md
  .env.example
  docs/              (these documents, diagrams)
  sdk/downshift/     (recorder, router)
  server/app/        (API, replay, evaluator, recommender)
  web/               (React UI)
  examples/research_agent/
  examples/coding_agent/
  tests/
```

8. Evaluation Plan

Test agents

- **A. Research agent:** answers a question using Tavily search and page summaries. Success = answer contains required facts from a fixed checklist.
- **B. Coding agent:** fixes failing tests in a small repository. Success = tests pass. Objective and hard to argue with.

Protocol

1. 5 baseline runs per agent on Ultra, each capped at 40 steps.
2. Replay all steps on Super and Nano.
3. Evaluate with objective checks and the blind judge.
4. Sweep the threshold at 0.8, 0.9 and 0.95.
5. Generate configs and run 5 validation runs per agent with the router.

Metrics

- Cost per run and cost reduction %
- Task success: baseline versus routed. Quality retained = routed success divided by baseline success.
- Latency per run
- % of steps downgraded
- Judge validation: manually review 20 random judgments (about 15 minutes) and report the agreement rate

Pass criteria (targets). 30% or more cost reduction with 95% or more quality retained on at least one agent. If this is not met, report the real numbers and the best threshold. An honest result beats a polished false one.

Credit control. Before the full run, estimate cost from token counts and prices. Run one agent end to end on a small sample first.

9. Demo Script (3 minutes)

- **0:00 to 0:20, Hook.** "Your agent runs on the most expensive model for every step. Most steps do not need it."
- **0:20 to 0:50, Record.** Run the agent on Ultra. Show the recorded timeline and the total cost.
- **0:50 to 1:30, Replay and judge.** Show steps replayed on Super and Nano through Token Factory. Show one objective pass and one judge verdict.
- **1:30 to 2:10, Report.** Timeline colored by recommended tier. Show the savings figure and the quality retained.
- **2:10 to 2:40, Proof.** Re-run with the router. Show real cost drop and task success on screen.
- **2:40 to 3:00, Close.** Name exactly how Nemotron tiers and Token Factory were used. Tagline.

Rules for the video. Record a fallback run first. Show real numbers only. Keep the terminal and UI readable at full-screen size. Upload as public to YouTube.

10. Submission Checklist

- Track selected: Coding and Agentic Engineering
- Project description: what it does, why, how it works
- Working demo URL
- Public YouTube video, 3 minutes or less
- Public repo with MIT license visible at the top, and README with setup and run steps
- README highlights: Nemotron usage, where Token Factory helped, other Nebius services used
- Written feedback on Nebius Token Factory, AI Cloud and NVIDIA tools (also eligible for the Most Valuable Feedback prize)
- If the project existed before the Submission Period, explain what changed. (If Downshift starts now, state that.)
- City Winner Award: select your city only if you attended an IRL event
- Read the official rules once, fully
- Submit at least 6 hours before the deadline

11. Access Checklist (do today)

- Nebius account created and Token Factory access confirmed
- API key stored in .env (never committed)
- Credits available and balance checked
- Exact Nemotron Ultra, Super and Nano model IDs confirmed by a test call
- Base URL, rate limits and pricing noted from the official docs
- Tavily API key
- GitHub repository created with MIT license (name: downshift-agent-router)
- YouTube channel ready for the video upload
- Devpost project created in draft

12. Build Schedule

- **25 Oct (today):** documents, access checklist, one test call to Nemotron.
- **26 Oct, Phase 1 Record:** SDK wrapper, storage, baseline agent A running on Ultra.
- **27 Oct, Phase 2 Replay and Judge:** parallel replays, objective checks, blind judge.
- **28 Oct, Phase 3 Report and Router:** recommendations, config export, validation run.
- **29 Oct, Phase 4 UI and Deploy:** timeline, cost chart, hosted demo, demo mode, agent B.
- **30 Oct, Phase 5 Submit:** README, video, Devpost. Target submission by 3 pm IST.

Cut order if time runs short: agent B, then Serverless Jobs, then UI polish. Never cut the validation run, because it is the proof.